

Requirement 6 - Performance Testing Report

Mục lục

- Requirement 6 - Performance Testing Report
 - Mục lục
 - Thông tin cá nhân & nhóm
 - Thông tin nhóm 11
 - 1. Tổng quan
 - 1.1. Các kỹ thuật kiểm thử hiệu năng
 - Chi tiết các kỹ thuật:
 - 1.2. System under test
 - 2. Môi trường kiểm thử
 - 2.1. Server
 - 2.2. Client chạy Test
 - 3. Quy trình thực hiện kiểm thử
 - Bước 1: Import Test Plan
 - Bước 2: Cấu hình Thread Group
 - Bước 3: Cấu hình Data-Driven Testing (CSV)
 - Bước 4: Cấu hình Authentication Cookie
 - Bước 5: Chạy Test và Xem Kết Quả
 - 4. Test Results
 - 4.1 Load test
 - 4.2. Stress test
 - 4.3. Spike Test
 - Phụ lục A: Hướng dẫn cài đặt Apache JMeter
 - Yêu cầu hệ thống
 - Cài đặt trên Linux
 - Phụ lục B: Hướng dẫn lấy Session Cookie (Authentication)
 - Cách 1: Sử dụng Script Python
 - Cách 2: Lấy thủ công qua Developer Tools (F12)
 - Lưu ý quan trọng về Cookie

Thông tin cá nhân & nhóm

- Họ tên: Phan Thanh Tiến
- MSSV: 22120368
- Nhóm 11.

Thông tin nhóm 11

- Thông tin thành viên:
 - Giang Đức Nhật - 22120252
 - Phan Thanh Tiến - 22120368
 - Nguyễn Bùi Vương Tiến - 22120370

◦ Lý Trọng Tín - 22120371

- Bảng phân công nhóm:

Tính năng	Mô tả	Thành viên
HR Administration	Quản trị hệ thống, cấu trúc tổ chức, user	Giang Đức Nhật
Employee Self-Service (ESS)	Cổng thông tin nhân viên tự phục vụ	Giang Đức Nhật
Recruitment	Tuyển dụng, theo dõi ứng viên	Phan Thanh Tiến
Performance Management	Đánh giá KPI, đề nghị xem xét, lưu lịch sử performance review	Phan Thanh Tiến
Reporting & Analytics	Báo cáo tùy chỉnh, xuất dữ liệu	Nguyễn Bùi Vương Tiễn
Time and Attendance	Chấm công, Timesheets	Nguyễn Bùi Vương Tiễn
Employee Management (PIM)	Quản lý hồ sơ nhân viên, báo cáo	Lý Trọng Tín
Leave Management	Quản lý ngày nghỉ, quy tắc nghỉ phép	Lý Trọng Tín

- Tính năng được phân công kiểm thử hiệu năng: **Recruitment (Create Candidate API)**

1. Tổng quan

Báo cáo này trình bày kết quả kiểm thử hiệu năng (Performance Testing) cho tính năng **Recruitment: Create Candidate** của hệ thống OrangeHRM. Kiểm thử được thực hiện sử dụng công cụ **Apache JMeter 5.6.3** với các kịch bản kiểm thử hiệu năng tiêu chuẩn.

1.1. Các thông số cần chú ý trong JMeter

Trong quá trình kiểm thử hiệu năng với Apache JMeter, chúng ta sử dụng ba thông số cấu hình chính để điều khiển tải tạo ra trên hệ thống:

- **Number of Threads:** số virtual users giả lập để truy cập hệ thống đồng thời. Mỗi thread tương ứng với một người dùng thực hiện các thao tác theo kịch bản đã định nghĩa.
- **Ramp-up Period:** Khoảng thời gian khởi tạo toàn bộ số lượng threads đã khai báo. Ví dụ, nếu sử dụng 100 threads với ramp-up 10 giây, JMeter sẽ khởi tạo dần các threads sao cho sau 10 giây, toàn bộ 100 threads đều đang hoạt động.
- **Loop Count:** Số lần lặp lại kịch bản kiểm thử của mỗi thread. Tổng số request gửi đi sẽ bằng (Number of Threads) * (Loop Count).

1.2. Các kỹ thuật kiểm thử hiệu năng

1.2.1 Load Testing

Load Testing là kỹ thuật kiểm thử được sử dụng để đánh giá hoạt động của hệ thống trong điều kiện tải bình thường.

Trong bài test này, kịch bản được thiết lập như sau:

- Lượng truy cập đồng thời: 100
- Thời gian khởi động: 2 giây.
- **Lặp lại 5 lần**, tạo ra tổng cộng 500 requests.

Mục tiêu chính của bài test:

- Xác định thời gian phản hồi (response time) và thông lượng (throughput) tiêu chuẩn.
- Đảm bảo hệ thống không phát sinh lỗi.
- Đảm bảo tài nguyên phần cứng (CPU, RAM) được sử dụng ở mức an toàn.

1.2.2 Stress Testing

Stress Testing đẩy hệ thống vượt qua giới hạn hoạt động bình thường để tìm điểm gãy (breaking point).

Thông số kịch bản test:

- Lượng truy cập đồng thời: 1000
- Thời gian khởi động: 2 giây.
- **Lặp lại 20 lần**, tổng cộng 20,000 requests.

Mục tiêu chính là xem xét:

- Khả năng chịu đựng tối đa của hệ thống.
- Sự suy giảm về hiệu năng (như response time tăng vọt).
- Thời điểm bắt đầu xuất hiện các lỗi quá tải hoặc connection timeout.

1.2.3 Spike Testing

Spike Testing mô phỏng tình huống lượng truy cập tăng đột biến trong thời gian cực ngắn, thường xảy ra trong các sự kiện đặc biệt. (Ví dụ thực tế như hàng loạt người vào đặt một món hàng khi nó vừa ra mắt)

Kịch bản thực hiện:

- Thiết lập **1500 người dùng** cùng ập vào hệ thống gần như tức thì.
- Thời gian ramp-up vòng vện trong **1 giây**.

Mục đích của bài test này:

- Đo lường độ trễ cực đại mà người dùng phải chịu đựng.
- Kiểm tra khả năng phục hồi của hệ thống sau lượng tải lớn như vậy (liệu server có bị sập hoàn toàn hay có thể giữ được trạng thái ổn định).

1.3. System under test

API Specifications:

- **Endpoint:** `POST /api/v2/recruitment/candidates`

- **Chức năng:** Tạo mới hồ sơ ứng viên trong hệ thống.
- **Authentication:** Session Cookie (`_orangehrm`)
- **Content-Type:** `application/json`

Request Payload Structure:

```
{
  "firstName": "TestUser1",
  "lastName": "Perf1",
  "email": "test.user.1@perf.com",
  "contactNumber": null,
  "keywords": null,
  "comment": null,
  "dateOfApplication": "2023-10-15",
  "consentToKeepData": false
}
```

Expected Response:

- **Success:** `200 OK` với JSON Object chứa thông tin ứng viên vừa tạo.
- **Failure:** `401 Unauthorized` nếu cookie hết hạn.

2. Môi trường kiểm thử

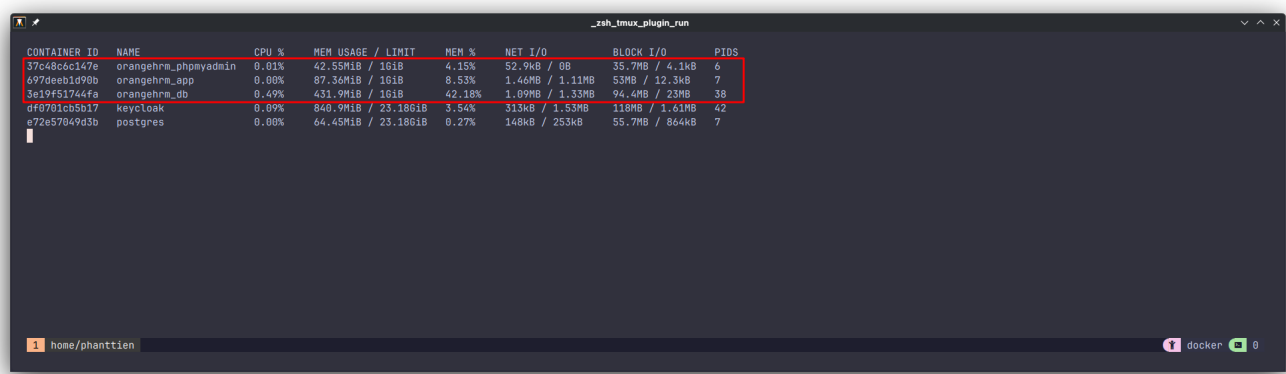
2.1. Server

Thành phần	Thông tin
Application	OrangeHRM 5.x (Open Source)
Base URL	<code>http://localhost:8080/web/index.php</code>
Web Server	Apache HTTP Server với PHP
Database	MySQL 8.x
Deployment	Docker Container (localhost)

Sau đây là thông tin của máy host server:

Thành phần	Thông tin
Operating System	Linux (Ubuntu 25.10)
CPU	Intel Core i5-1135G7
RAM	24GB DDR4
Disk	SSD 512GB

Tuy nhiên, mỗi container sẽ được giới hạn với **1 core CPU và 1GB RAM**.



Hình 1: Cấu hình giới hạn container 1 CPU / 1GB RAM.

2.2. Client chạy Test

Thành phần	Thông tin
Operating System	Linux (Ubuntu/Debian)
Test Tool	Apache JMeter 5.6.3
Java Version	OpenJDK 21+
Network	Localhost
Memory Allocation	JVM Heap: 2GB (-Xms2g -Xmx4g)

3. Quy trình thực hiện kiểm thử

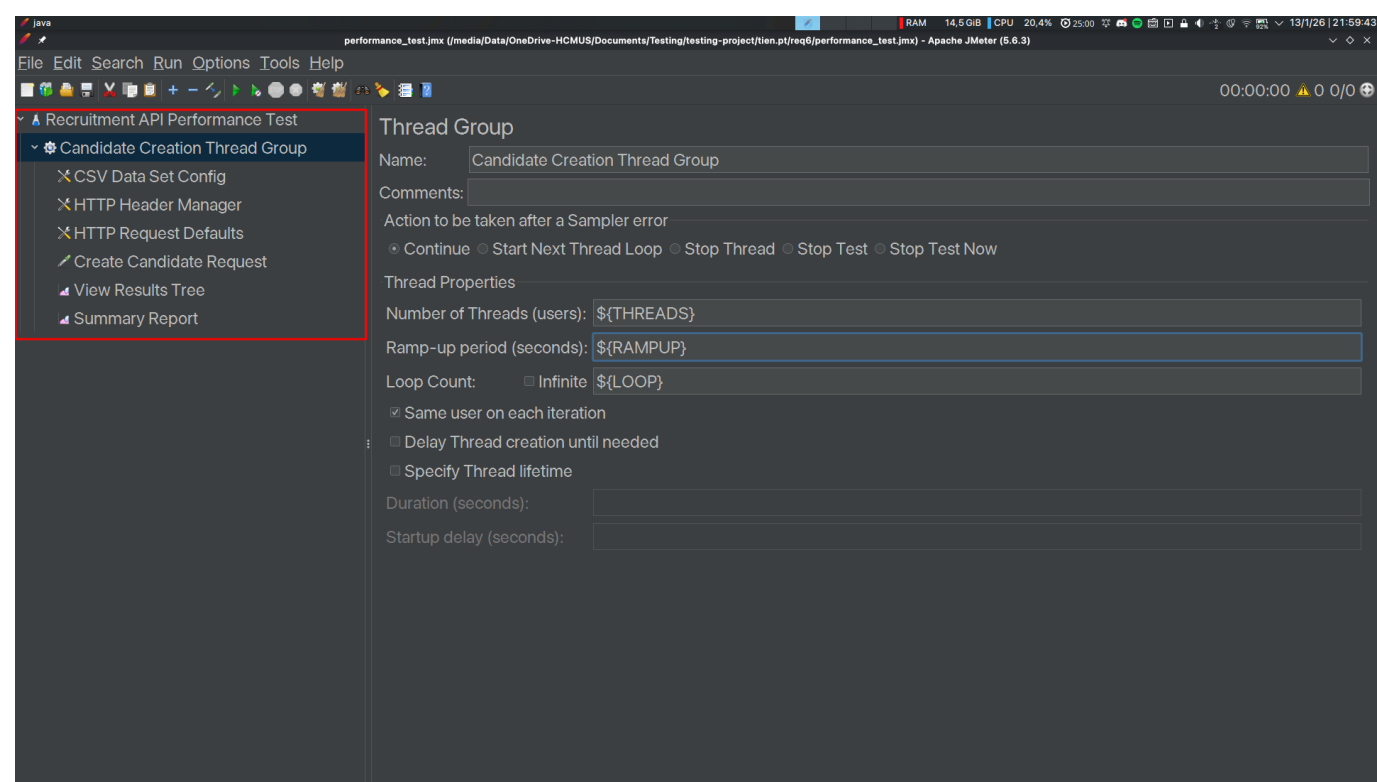
Chúng ta sẽ sử dụng JMeter GUI cho phần kiểm thử này. JMeter cũng hỗ trợ CLI, nhưng sử dụng GUI sẽ có phần report chi tiết và dễ xem hơn.

Bước 1: Import Test Plan

- 1. Clone repository về máy:

```
git clone <repository-url>
cd tien.pt/req6
```

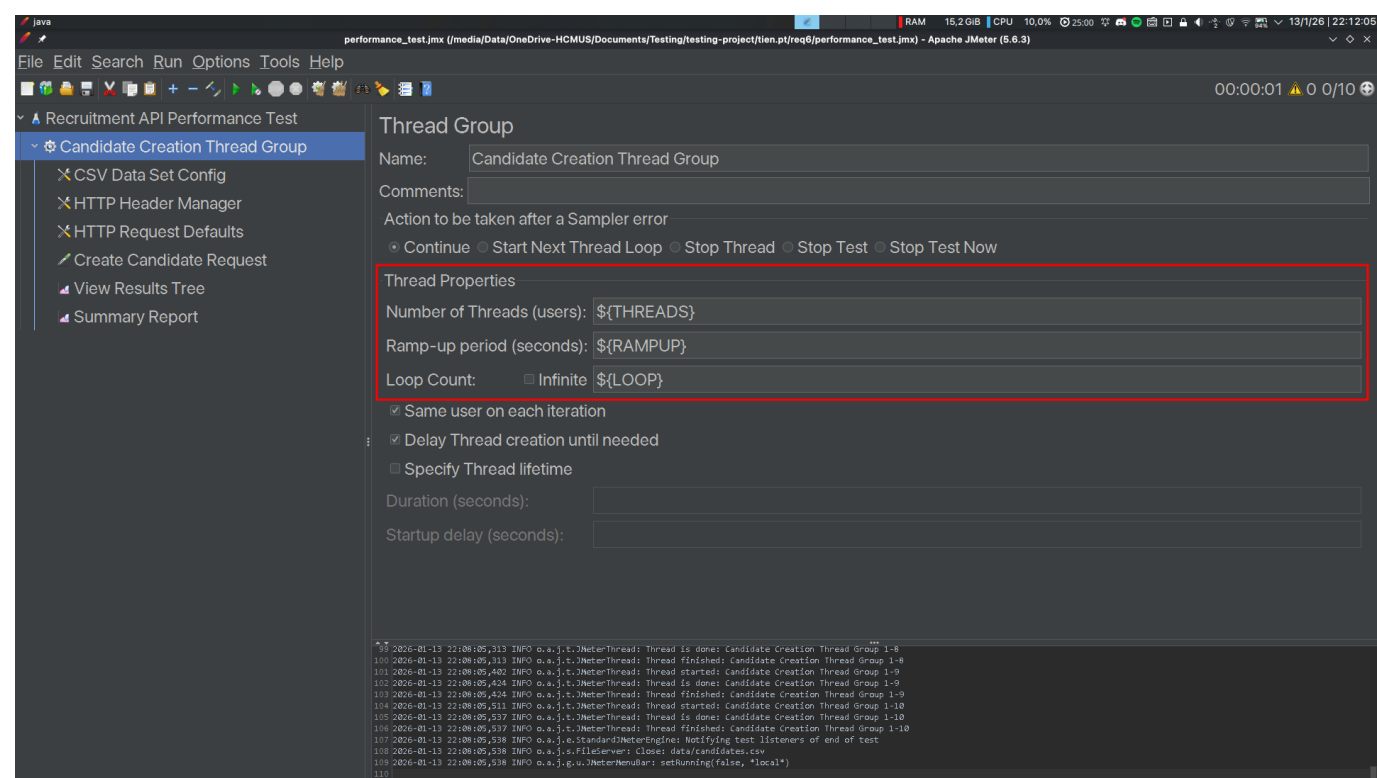
- 2. Mở JMeter và import file `performance_test.jmx` bằng cách:
 - File → Open → Chọn `performance_test.jmx`



Hình 2: Cấu trúc Test Plan với các thành phần: CSV Data Set Config, HTTP Header Manager, HTTP Request Defaults, Create Candidate Request, và Summary Report.

Bước 2: Cấu hình Thread Group

Điều chỉnh các tham số trong Thread Group theo kịch bản test:

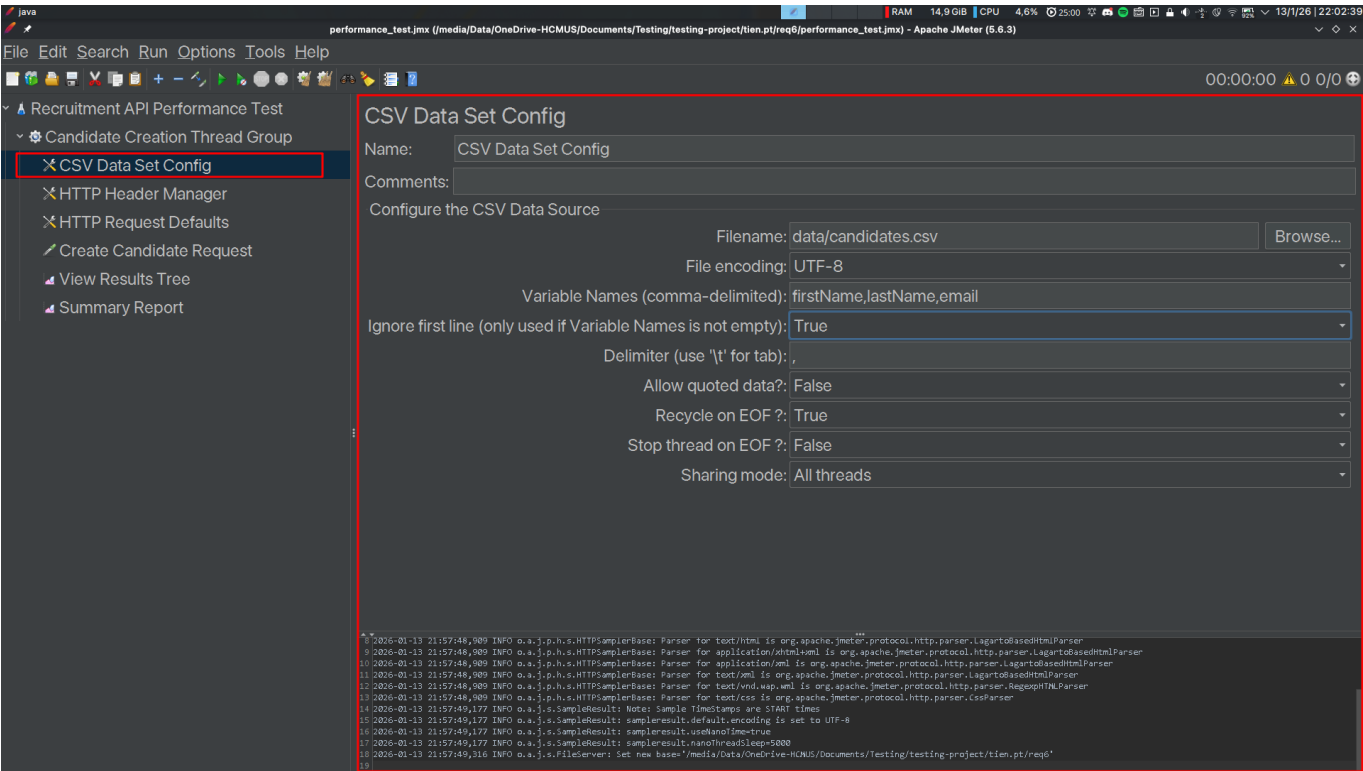


Hình 3: Cấu hình Thread Group với các biến `${THREADS}`, `${RAMPUP}`, `${LOOP}` cho phép linh hoạt thay đổi.

Tham số	Ý nghĩa
Number of Threads	Số lượng virtual users đồng thời
Ramp-up Period	Thời gian để khởi tạo tất cả threads (giây)
Loop Count	Số lần lặp cho mỗi thread

Bước 3: Cấu hình Data-Driven Testing (CSV)

Dữ liệu ứng viên được tham số hóa thông qua file CSV để đảm bảo mỗi request tạo một ứng viên với thông tin khác nhau.



Hình 4: Cấu hình CSV Data Set Config để đọc dữ liệu từ file data/candidates.csv.

Cấu trúc file data/candidates.csv:

```
firstName,lastName,email
TestUser1,Perf1,test.user.1@perf.com
TestUser2,Perf2,test.user.2@perf.com
TestUser3,Perf3,test.user.3@perf.com
...
TestUser50,Perf50,test.user.50@perf.com
```

The screenshot shows a LibreOffice Calc spreadsheet titled 'candidates.csv'. The table has three columns: 'firstName', 'lastName', and 'email'. It contains 18 rows of test user data. The Properties sidebar on the right is open, showing the 'Style' section with 'Default' selected, 'Character' section with 'Liberation Sans' font and '10 pt' size, and 'Number Format' section with 'General' format and '1' leading zeros.

firstName	lastName	email
TestUser1	Perf1	test.user.1@perf.com
TestUser2	Perf2	test.user.2@perf.com
TestUser3	Perf3	test.user.3@perf.com
TestUser4	Perf4	test.user.4@perf.com
TestUser5	Perf5	test.user.5@perf.com
TestUser6	Perf6	test.user.6@perf.com
TestUser7	Perf7	test.user.7@perf.com
TestUser8	Perf8	test.user.8@perf.com
TestUser9	Perf9	test.user.9@perf.com
TestUser10	Perf10	test.user.10@perf.com
TestUser11	Perf11	test.user.11@perf.com
TestUser12	Perf12	test.user.12@perf.com
TestUser13	Perf13	test.user.13@perf.com
TestUser14	Perf14	test.user.14@perf.com
TestUser15	Perf15	test.user.15@perf.com
TestUser16	Perf16	test.user.16@perf.com
TestUser17	Perf17	test.user.17@perf.com
TestUser18	Perf18	test.user.18@perf.com

Hình 5: Cấu hình HTTP Request Defaults (domain, port).

Bước 4: Cấu hình Authentication Cookie

1. Lấy session cookie hợp lệ (Tham khảo Phụ lục B).
2. Cập nhật biến **COOKIE** trong User Defined Variables hoặc qua CLI parameter.

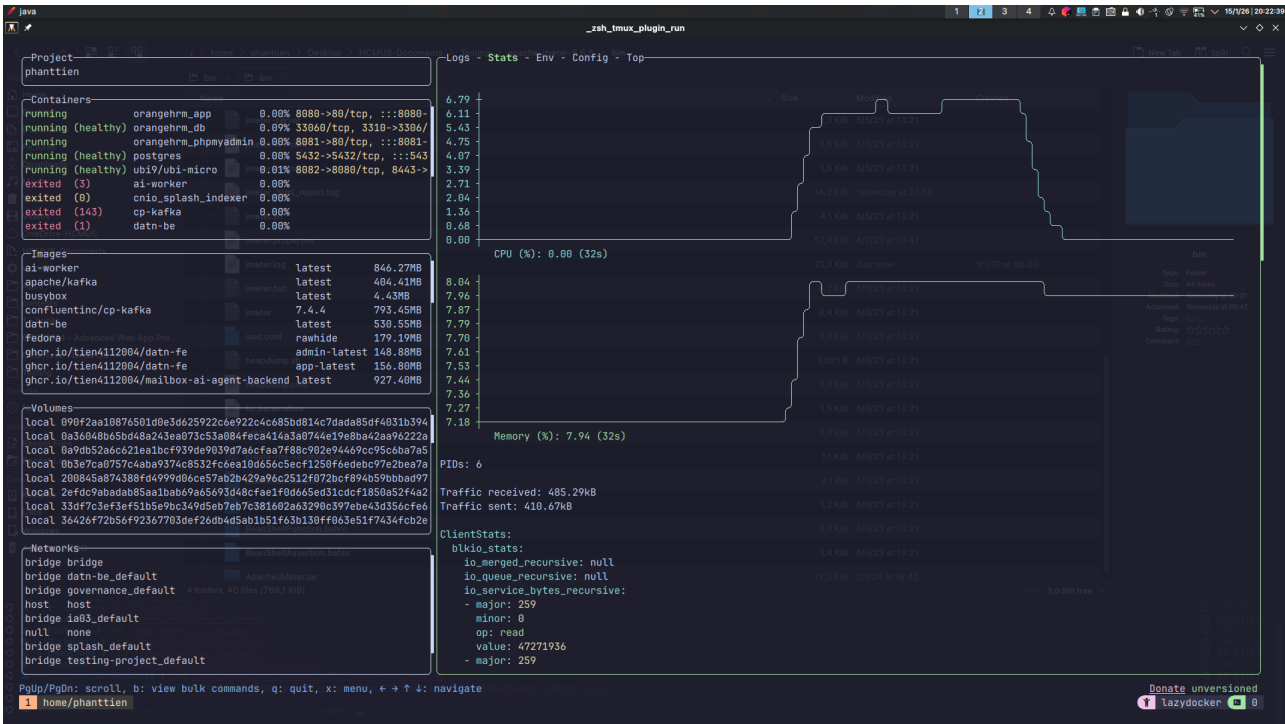
Bước 5: Chạy Test và Xem Kết Quả

Về kết quả chạy thực tế sẽ được nêu ở phần 4 bên dưới.

4. Test Results

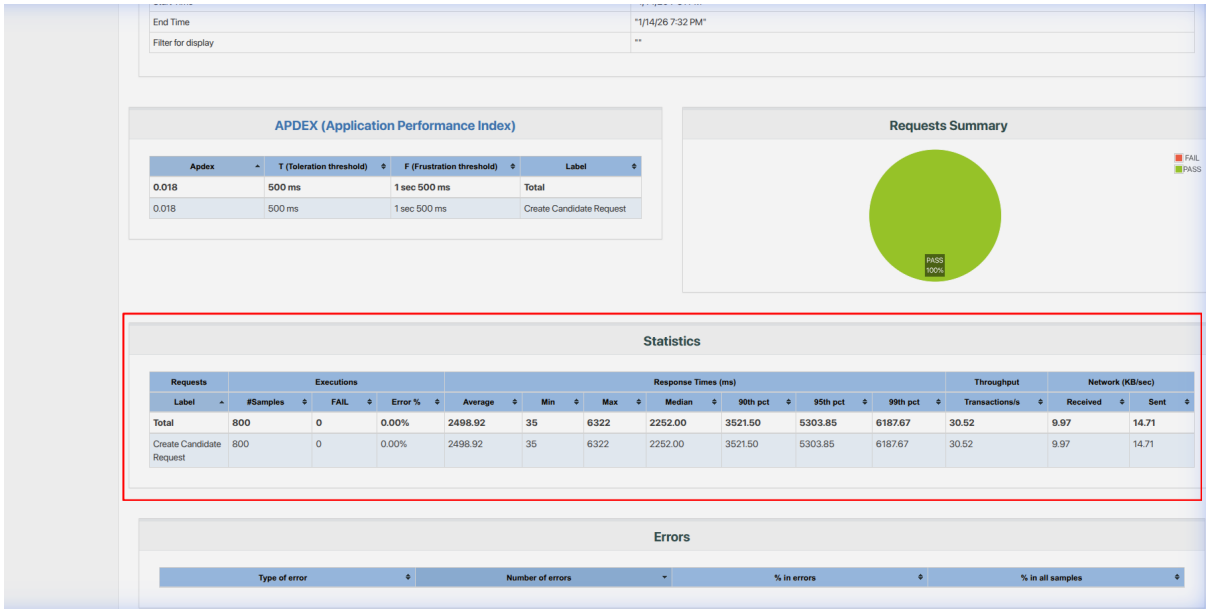
4.1 Load test

- Cấu hình bộ test:
 - Threads: 100
 - Rampup time: 2 seconds
 - Loop: 5
- Kết quả:
 - Hệ thống hoạt động ổn định dưới tải (100 concurrent users), không có dấu hiệu quá tải CPU/RAM. CPU không được sử dụng hết. **[Lưu ý: Số phần trăm hiển thị trên công cụ là của tổng 8 CPU -> 1 core sử dụng tối đa 12.5%]**



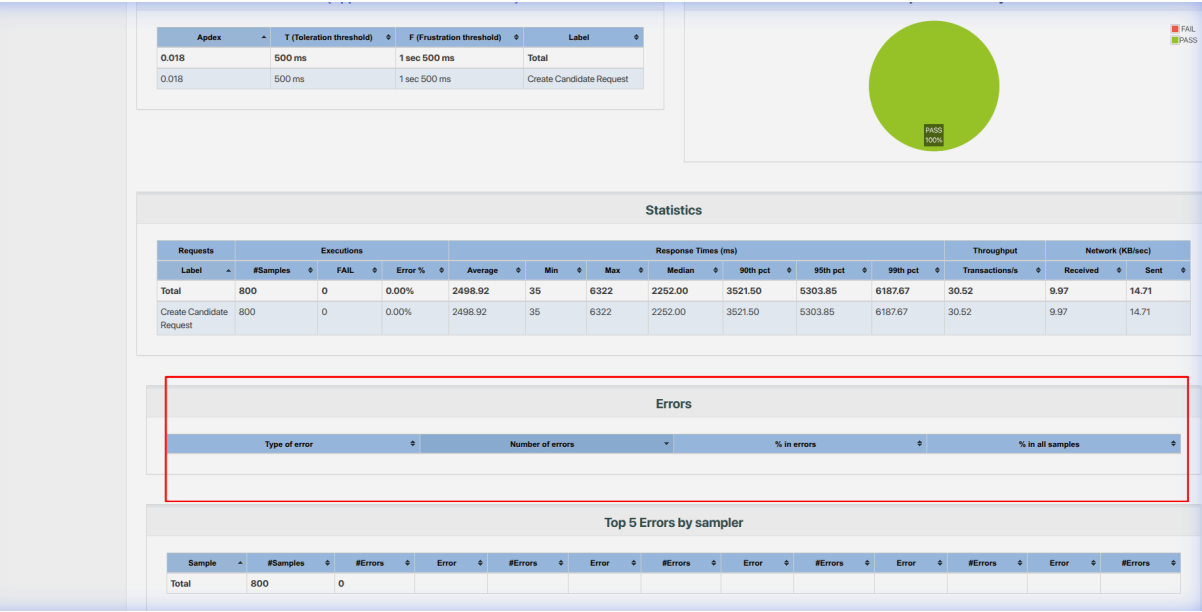
Hình 6: Sử dụng tài nguyên hệ thống ổn định.

- Summary report: Quan sát thấy các thông tin sau:
 - Số lượng request: 800
 - Thời gian phản hồi trung bình (Avg Response Time): **2,499 ms** (~2.5s)
 - Thời gian phản hồi tối đa (Max Response Time): **6,322 ms** (~6.3s)
 - Tỷ lệ lỗi (Error Rate): **0.00%**
 - Thông lượng (Throughput): **30.5 req/sec**



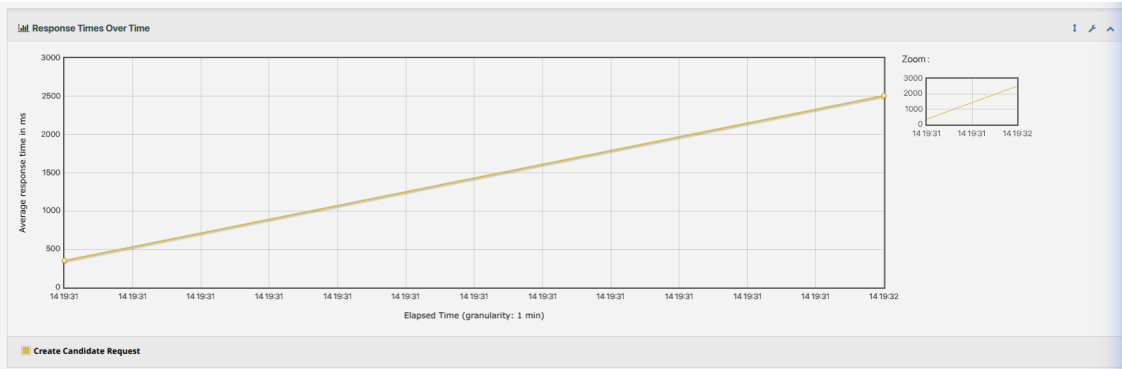
Hình 7: Dashboard Summary - Load Test Results.

- Quan sát danh sách các lỗi, ta thấy danh sách lỗi rỗng, cho thấy hệ thống xử lý tốt ở mức tải này.



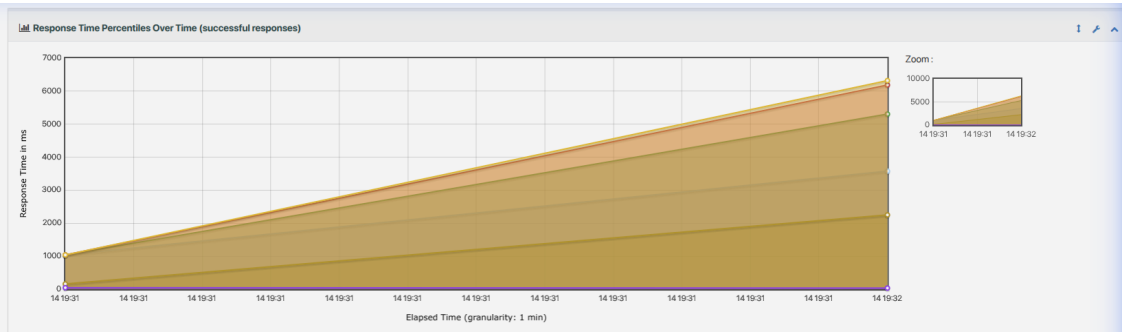
Hình 8: Danh sách lỗi Load Test (0%).

- Graph (Report đầy đủ: <https://github.com/ltt204/database-testing-presentation/tree/main/22120368/req6/load>)
 - Response time: Response time dao động quanh mức trung bình 2.8s, không có sự tăng đột biến nào đáng kể.



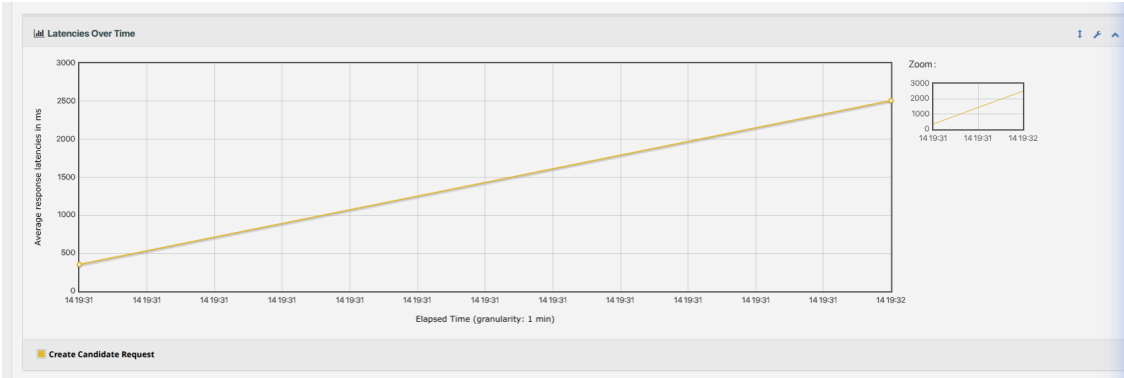
Hình 9: Biểu đồ Response Time theo thời gian (Load Test).

- Response time percentile: Các đường P90, P95, P99 nằm khá gần nhau và gần với mức trung bình, chứng tỏ độ ổn định cao.



Hình 10: Biểu đồ Percentiles (Load Test).

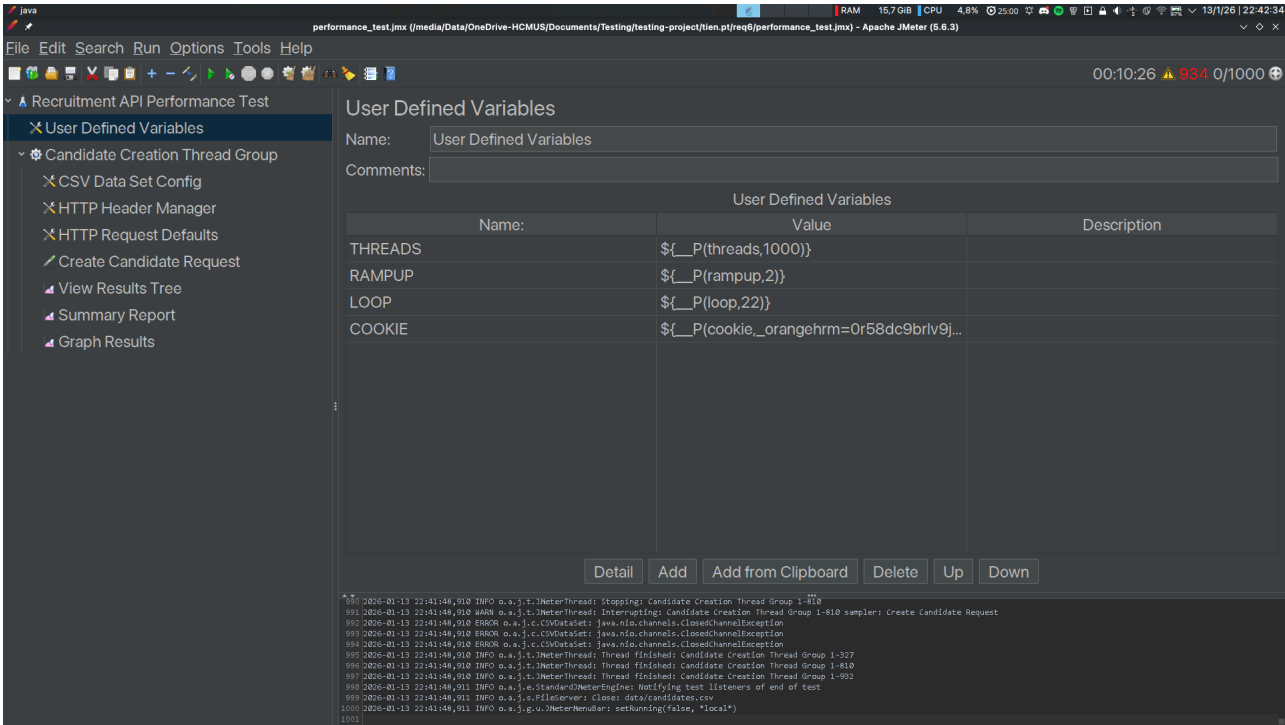
- Latency: Latency duy trì ở mức thấp và ổn định trong suốt quá trình test.



Hình 11: Biểu đồ Latency (Load Test).

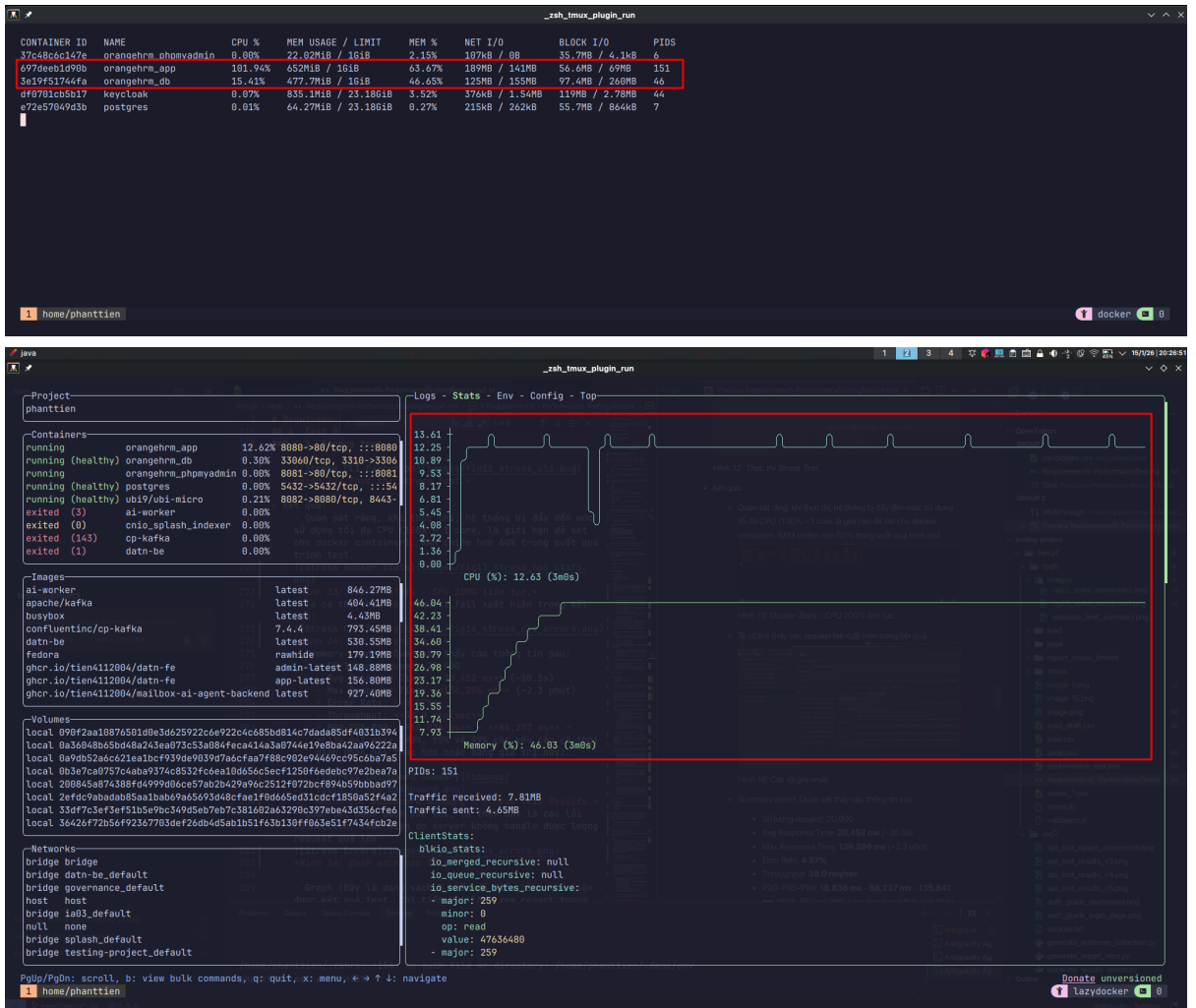
4.2. Stress test

- Cấu hình bộ test:
 - Threads: 1000
 - Rampup time: 2 seconds
 - Loop: 20



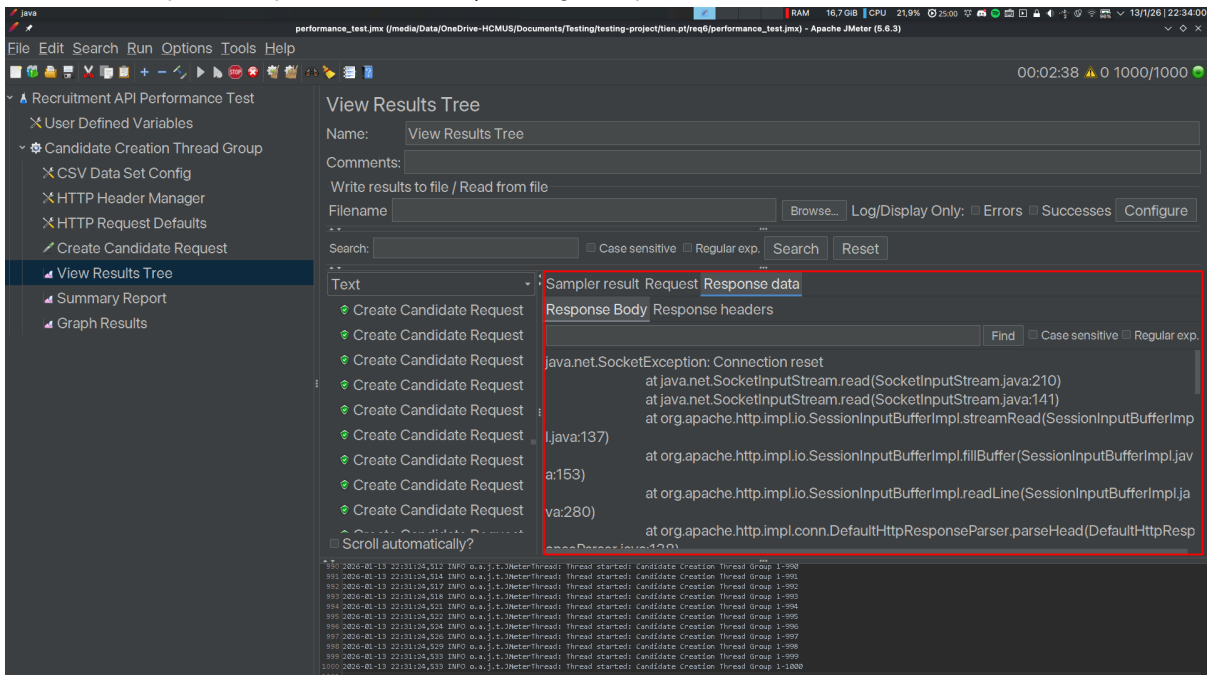
Hình 12: Thực thi Stress Test.

- Kết quả:
 - Quan sát rằng, khi thực thi, hệ thống bị đẩy đến mức sử dụng tối đa CPU (100% ~ 1 core, là giới hạn đã set cho docker container). RAM chiếm hơn 60% trong suốt quá trình test.



Hình 13: Docker Stats - CPU 100% liên tục.

- Ta có thể thấy các request fail xuất hiện trong kết quả:

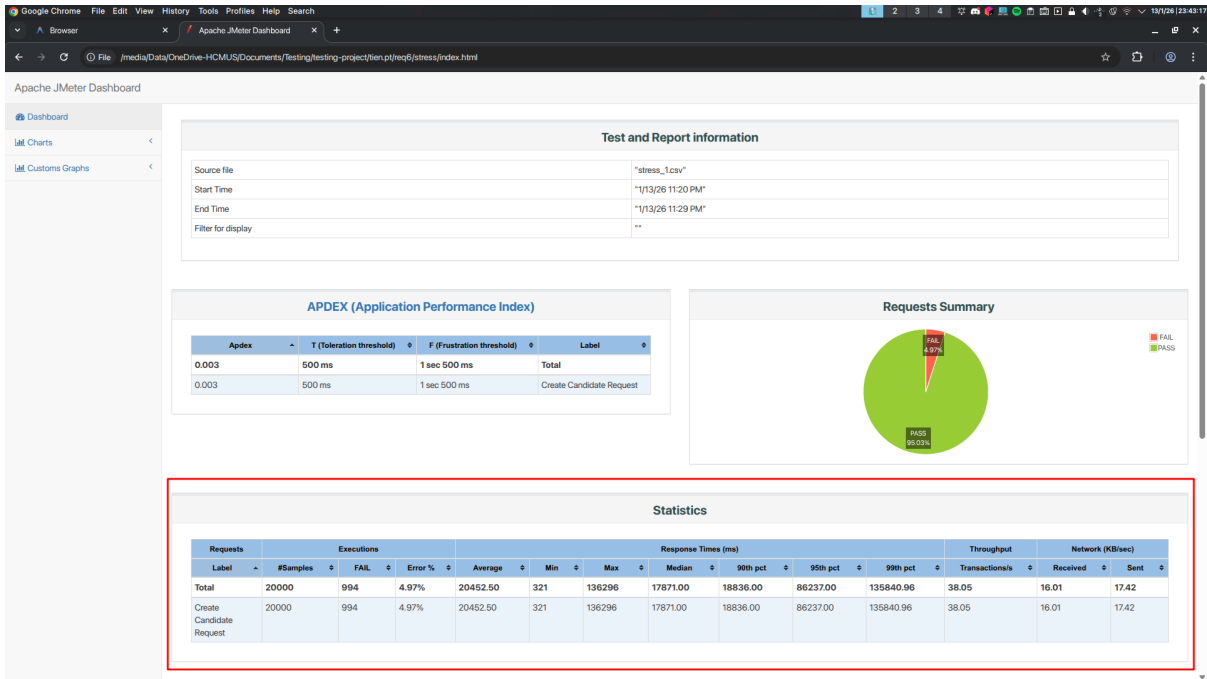


Hình 14: Các lỗi ghi nhận.

- Summary report: Quan sát thấy các thông tin sau:

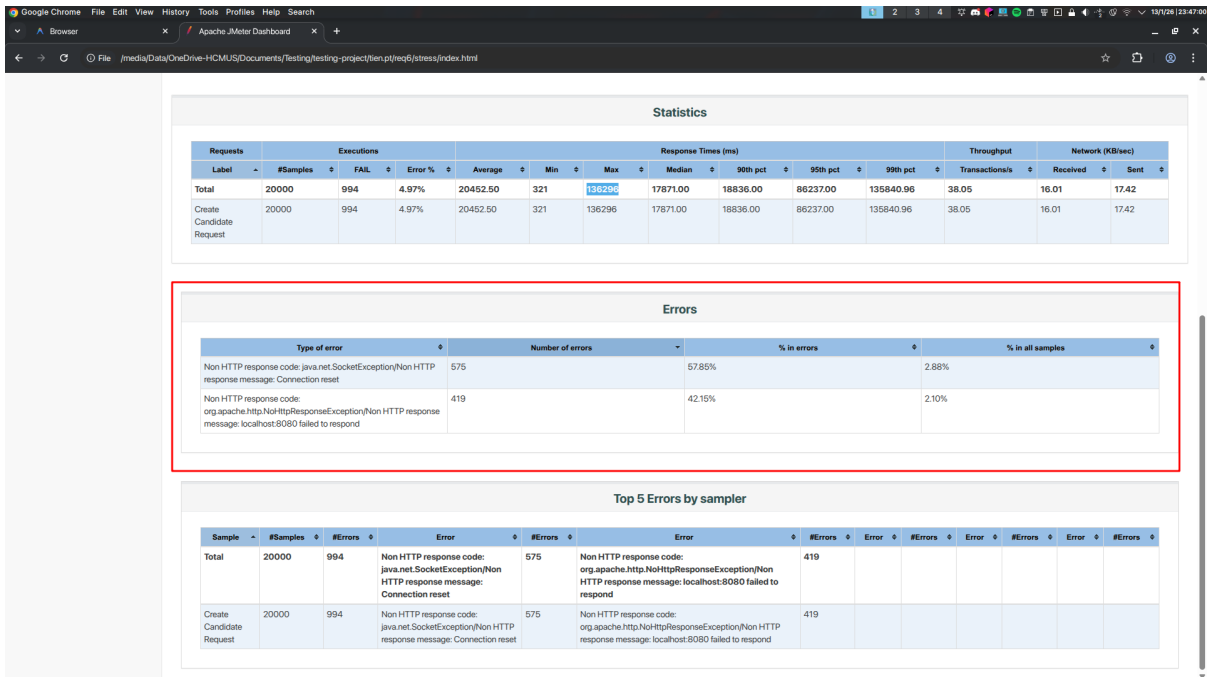
- Số lượng request: 20,000

- Avg Response Time: **20,452 ms** (~20.5s)
- Max Response Time: **136,296 ms** (~2.3 phút)
- Error Rate: **4.97%**
- Throughput: **38.0 req/sec**
- P90-P95-P99: **18,836 ms - 86,237 ms - 135,841 ms** (90%, 95% và 99% các yêu cầu có thời gian phản hồi thấp hơn hoặc bằng giá trị này).



Hình 15: Dashboard Summary - Stress Test Results.

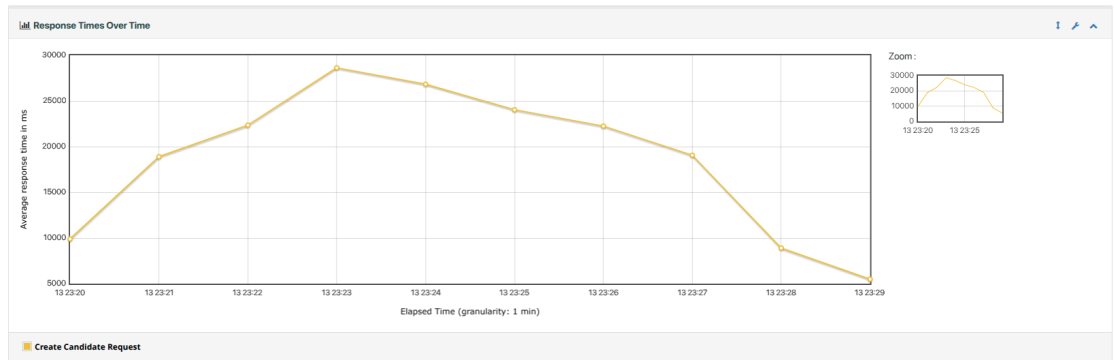
- Quan sát danh sách các lỗi, ta thấy đây là các lỗi connection, xuất hiện do server không handle được lượng request quá lớn



Hình 16: Danh sách các lỗi

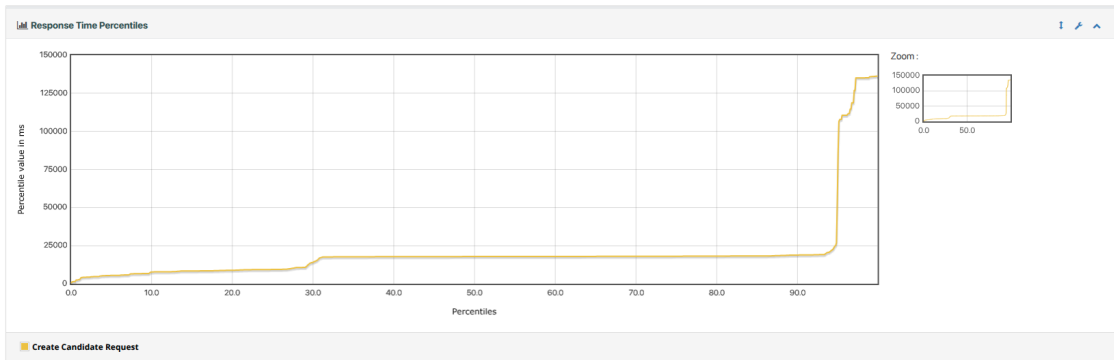
- Graph (Đây là danh sách 3 biểu đồ diễn hình thể hiện được kết quả test. Report đầy đủ: <https://github.com/Itt204/database-testing-presentation/tree/main/22120368/req6/stress>)

- Response time: Response time tăng cao ở thời điểm giữa, là thời điểm server đã chịu tải nặng một thời gian tương đối để khiến server quá tải. Sau thời điểm đỉnh, khi các request đã fail hoặc được giải quyết phần nào, ta thấy response time giảm đáng kể.



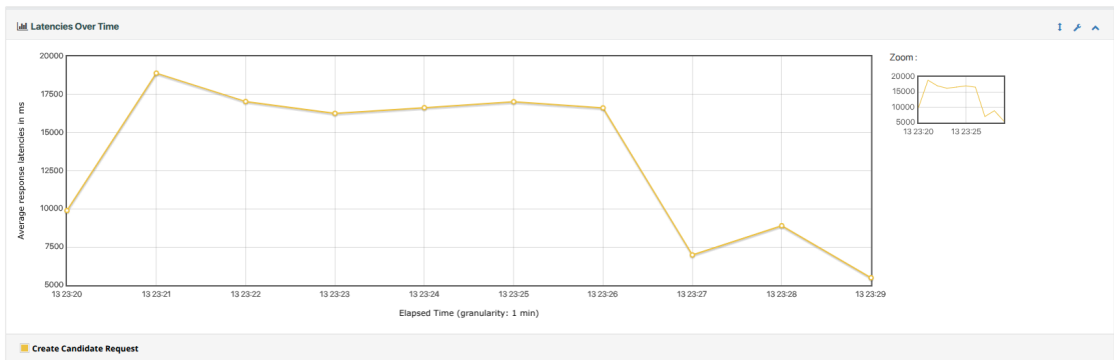
Hình 17: Biểu đồ Response Time (Stress Test).

- Response time percentile: Quan sát thấy P90, P95 và P99 lệch rất đáng kể. Điều này cho thấy khi server quá tải sẽ gây ảnh hưởng lớn đến response time.



Hình 18: Biểu đồ Percentiles phân tách rõ rệt khi quá tải.

- Latency: Quan sát thấy latency từ thấp ở thời điểm bắt đầu, tăng mạnh lên khi server bắt đầu quá tải, và giảm dần khi các request được giải quyết.

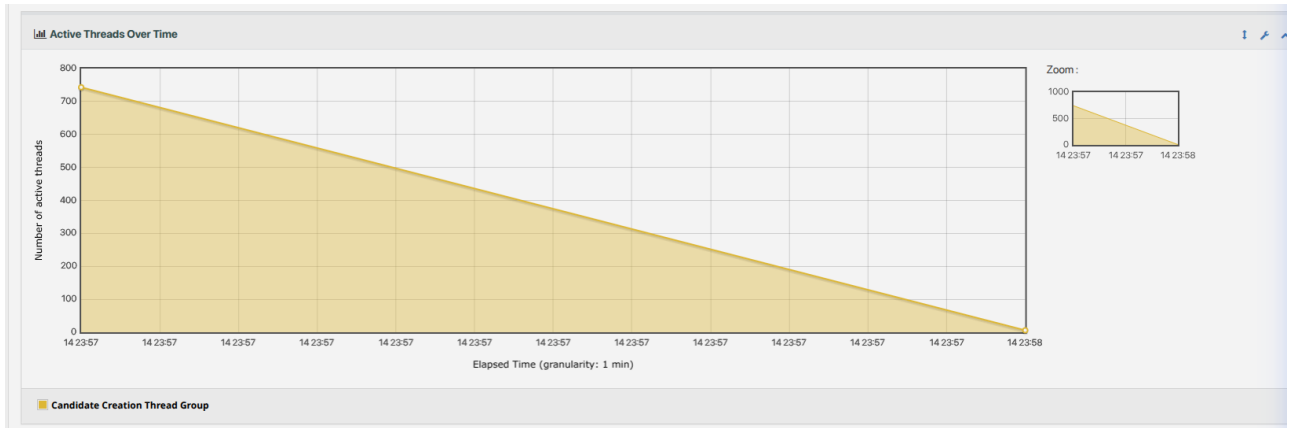


Hình 19: Biểu đồ Latency tăng mạnh (Stress Test).

4.3. Spike Test

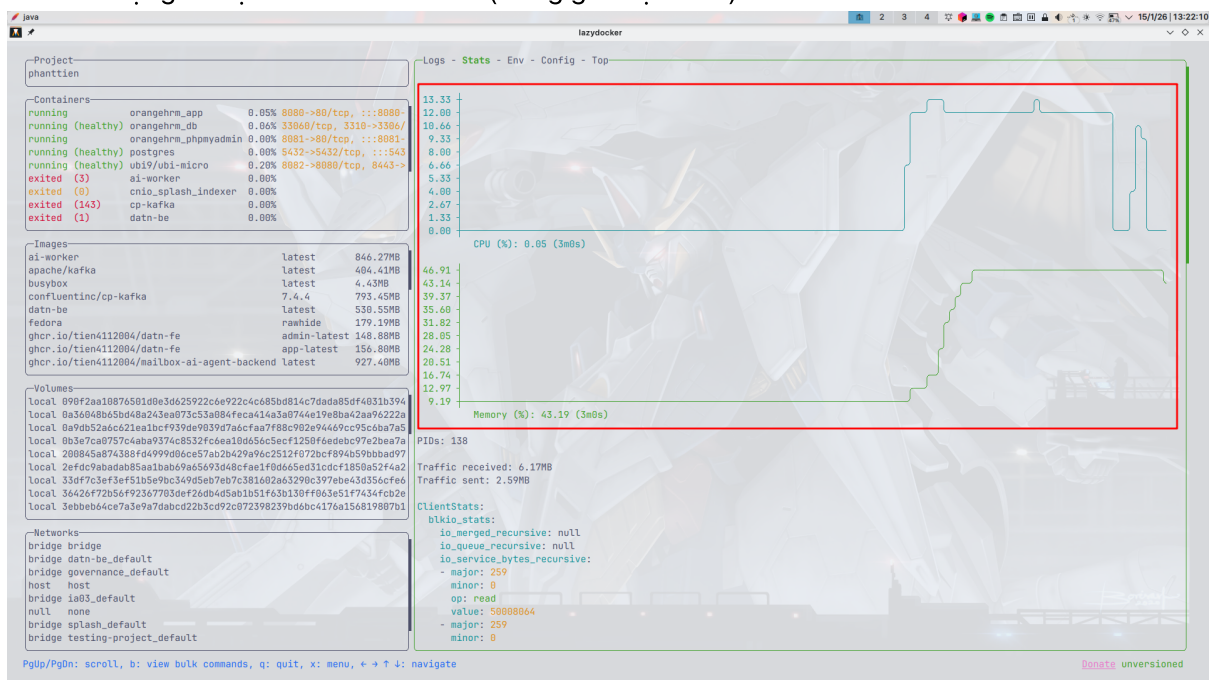
- Cấu hình bộ test:
 - Threads: 1500
 - Rampup time: 1 second (1500 user cùng đổ vào hệ thống trong 1s)
 - Loop: 1

- Quan sát chart dưới, dễ dàng thấy rằng lượng request tăng cao ở thời gian đầu mô phỏng cho trường hợp người dùng đổ dồn vào hệ thống trong thời gian ngắn. (Gần 800 thread ở thời gian đầu, giảm dần khi các request được xử lý).

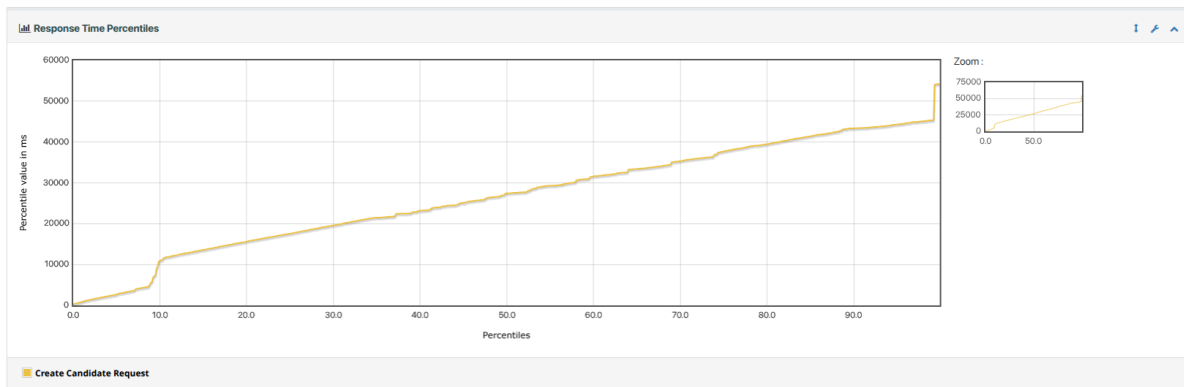


Hình 20: Lượng user (Active Threads) tăng dựng đứng trong 1s.

- Tài nguyên hệ thống sử dụng:
 - CPU liên tục ở mức 100% (1 core).
 - RAM sử dụng ổn định ở mức ~600MB (trong giới hạn 1GB).

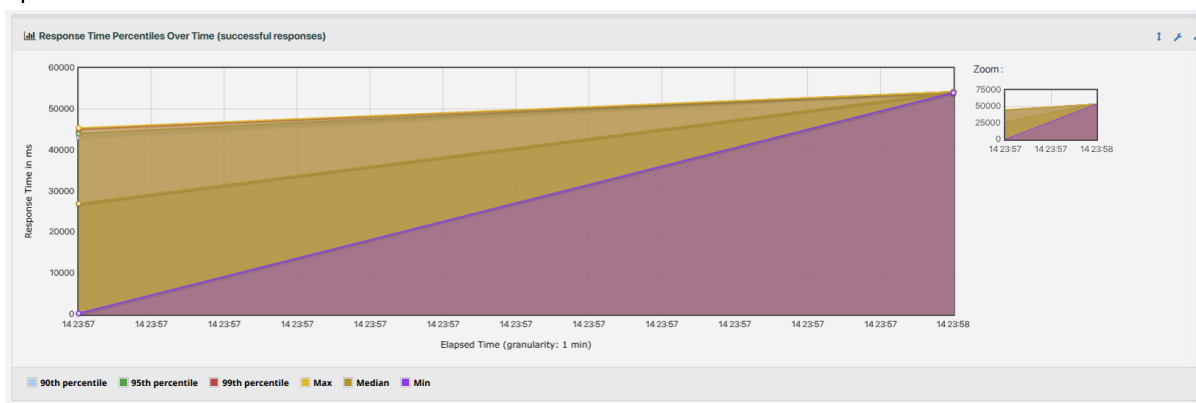


- Kết quả:
 - Report đầy đủ: <https://github.com/Itt204/database-testing-presentation/tree/main/22120368/req6/peak>
 - Hệ thống vẫn duy trì được hoạt động và xử lý toàn bộ request mà **không có lỗi** (0% Error Rate).
 - Tuy nhiên, thời gian phản hồi bị ảnh hưởng nghiêm trọng do số lượng request đến cùng lúc quá lớn. Độ chênh lệch giữa request nhanh nhất và chậm nhất là rất đáng kể, nhanh nhất là 298ms, trong khi chậm nhất 54251ms, khoảng hơn 54s.



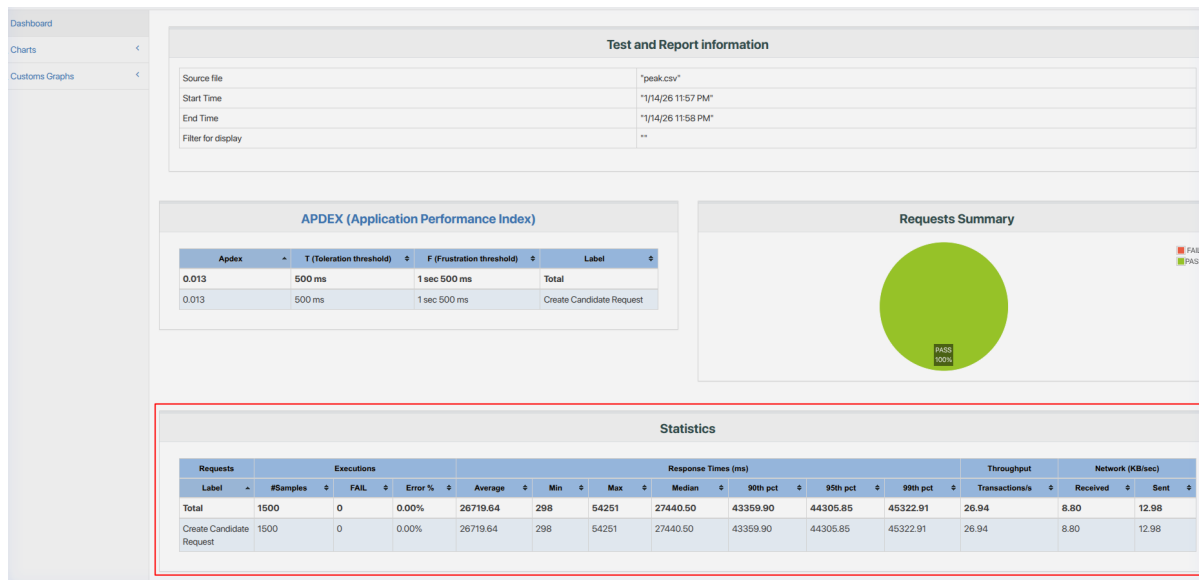
Hình 21: Response Time tăng vọt tương ứng với Spike.

- Response time percentile: Các đường P90, P95, P99 tăng vọt và phân tách rõ rệt, chứng tỏ người dùng ở nhóm tail (nhóm chậm nhất) chịu độ trễ cực lớn (lên tới 45s-50s) trong thời điểm spike.



Hình 22: Percentiles Graph - P99 chạm ngưỡng ~50s.

- Summary report: Quan sát thấy các thông tin sau:
 - Số lượng request: 1500
 - Thời gian phản hồi trung bình (Avg Response Time): **26,720 ms** (~26.7s)
 - Thời gian phản hồi tối đa (Max Response Time): **54,251 ms** (~54.3s)
 - Tỷ lệ lỗi (Error Rate): **0.00%**
 - Thông lượng (Throughput): **26.9 req/sec**



Hình 23: Dashboard Summary - Spike Test Results.

Phụ lục A: Hướng dẫn cài đặt Apache JMeter

Yêu cầu hệ thống

- Java JDK 11 hoặc cao hơn
- Cấu hình cho JVM:

```
# Chỉnh sửa file jmeter (hoặc jmeter.bat trên Windows)
# Tăng heap size cho JVM
HEAP="-Xms2g -Xmx4g -XX:MaxMetaspaceSize=512m"
```

Cài đặt trên Linux

```
# 1. Cài đặt Java (nếu chưa có)
sudo apt update
sudo apt install openjdk-11-jdk -y

# 2. Kiểm tra Java version
java -version

# 3. Tải Apache JMeter
wget https://downloads.apache.org/jmeter/binaries/apache-jmeter-5.6.3.tgz

# 4. Giải nén
tar -xzf apache-jmeter-5.6.3.tgz

# 5. Thêm vào PATH (tùy chọn)
echo 'export PATH=$PATH:~/apache-jmeter-5.6.3/bin' >> ~/.bashrc
source ~/.bashrc

# 6. Chạy JMeter GUI
jmeter
```

```
# 7. Hoặc chạy Non-GUI mode
jmeter -n -t test.jmx -l results.jtl
```

Phụ lục B: Hướng dẫn lấy Session Cookie (Authentication)

Do hệ thống OrangeHRM sử dụng **HttpOnly Cookie** (`_orangehrm`) để bảo mật, việc lấy cookie này thông qua JavaScript console (`document.cookie`) là không thể. Dưới đây là 2 cách để lấy giá trị này.

Cách 1: Sử dụng Script Python

Script Python đã được phát triển để tự động login và lấy session cookie một cách nhanh chóng. Để sử dụng, chạy lệnh sau:

```
# Chạy script
python3 ../req7/get_auth_cookie.py

# Output mẫu
# VALID_COOKIE: abc123xyz...
```

Sau đó sử dụng cookie trong lệnh JMeter:

```
jmeter -n -t performance_test.jmx -Jcookie="orangehrm=abc123xyz..." -l
results.jtl
```

Cách 2: Lấy thủ công qua Developer Tools (F12)

1. Mở trang Login OrangeHRM và mở DevTools (F12), chuyển sang tab **Network**.
2. Thực hiện đăng nhập thành công.
3. Trong danh sách Network, tìm request có tên `validate` (Method POST) hoặc `index`.
4. Click vào request, chọn tab **Headers**.
5. Tìm phần **Response Headers** → Copy giá trị của `_orangehrm` (bỏ phần `; path=/...`).

Lưu ý quan trọng về Cookie

- Session cookie có TTL 30 phút nếu không hoạt động.
- Cần lấy cookie mới trước mỗi lần chạy test nếu session đã hết hạn.
- Kiểm tra response code `401 Unauthorized` để biết cookie đã hết hạn.